# XSS-Labs Report

**Target:** Rezy-Dev XSS-Labs (local instance, localhost:8000)
**Scope:** Labs 1–16 (excluding CSP Bypass labs)
**Date:** October 29th , 2026

# Table of Contents


1 Lab 1 - Reflected XSS in HTML Context ..... 1
2 Lab 2 - Reflected XSS in Attribute Context ..... 2
3 Lab 3 - Reflected XSS in JavaScript Context ..... 3
4 Lab 4 - Reflected XSS in Input Value ..... 5
5 Lab 5 - Reflected XSS in URL Context ..... 6
6 Lab 6 - Reflected XSS with Basic Filtering (Filter Bypass) ..... 7
7 Lab 7 -"Impossible XSS" (Properly Mitigated) ..... 9
8 Lab 8 - Reflected XSS with Weak Custom Filters (Blacklist Bypass) ..... 10
9 Lab 9 - Reflected XSS with Moderate Custom Filters (Blacklist Bypass) ..... 11
10 Lab 10 - Reflected XSS in JS Context with Basic Filtering ..... 13
11 Lab 11 - Stored XSS ..... 14
12 Lab 12 - Stored XSS with Basic Filtering ..... 16
13 Lab 13 - DOM-Based XSS ..... 18
14 Lab 14 - Advanced DOM-Based XSS ..... 19
15 Lab 15 - Blind XSS via Stored User Input ..... 21
16 Lab 16 - Stored Blind XSS via User-Agent Header ..... 23

# 1 Lab 1 - Reflected XSS in HTML Context

**Vulnerable file:** lab1.php

**Vulnerable code:**

```
$search = $_GET['q'] ?? '';
$output = $search ? "<p>Results for: <b>$search</b></p>" : '';
```

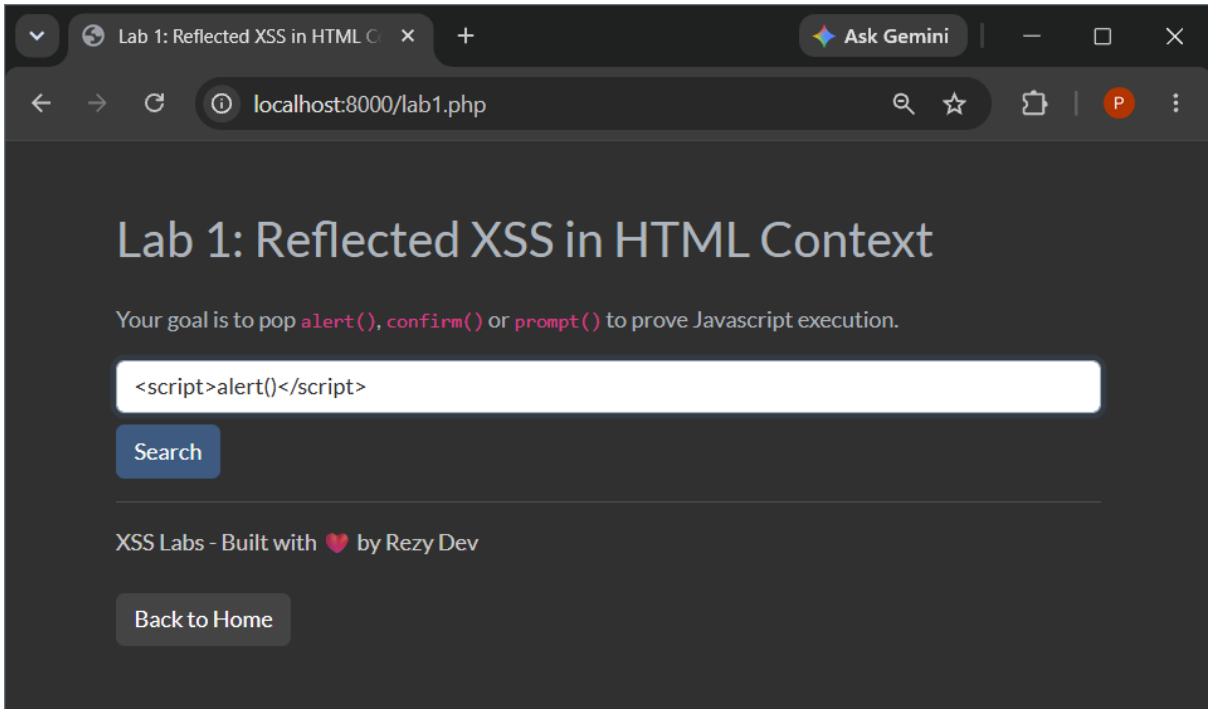
**Root cause:** User input ($_GET['q']) is concatenated directly into the HTML response inside a <b> tag with no output encoding (no htmlspecialchars()), allowing injected markup to be parsed and executed by the browser.

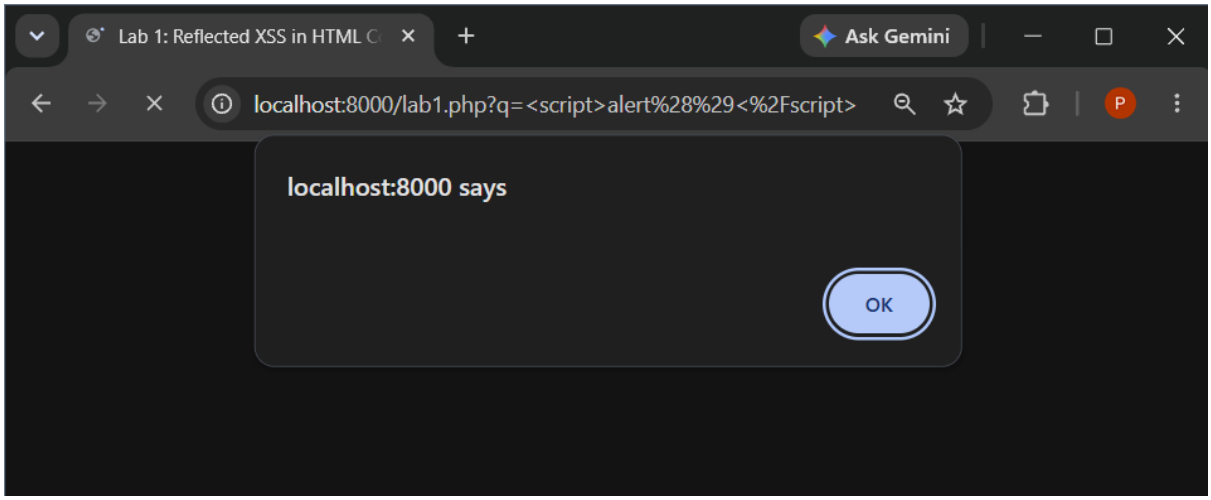
**Payload used:** <script>alert()</script>

**Request:** GET /lab1.php?q=<script>alert()</script>

**Result:** JavaScript executed successfully — alert() dialog popped, confirming reflected XSS.

**Evidence:**






## 2 Lab 2 - Reflected XSS in Attribute Context

**Vulnerable file:** lab2.php

**Vulnerable code:**

```
$name = $_GET['name'] ?? '';
……..
<img src="avatar.jpg" alt="$name" class="mt-3"/>
```

**Root cause:** User input ($_GET['name']) is reflected unencoded inside the double-quoted alt attribute of an <img> tag. No htmlspecialchars() or attribute encoding is applied, allowing an attacker to close the attribute and tag, then inject arbitrary HTML/JS into the document body.

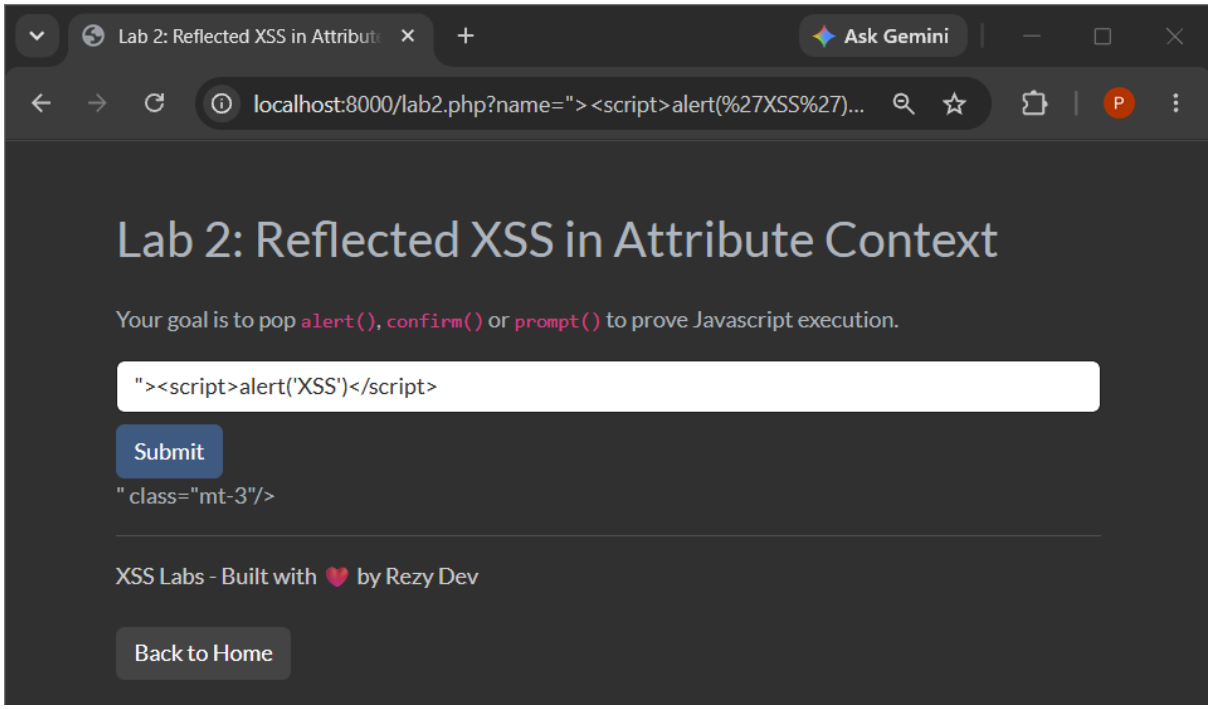
**Payload used:** "><script>alert('XSS')</script>

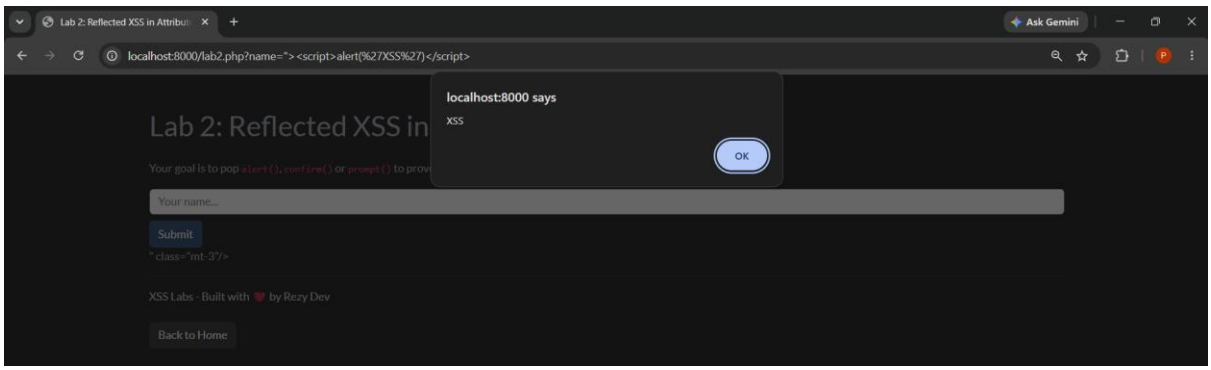
**Request**: GET /lab2.php?name="><script>alert('XSS')</script>

**Breakout mechanics:** " closes the alt attribute -> closes the <img> tag -> fresh <script> element injected into the HTML body, executed by the browser.

**Result:** JavaScript executed successfully — alert('XSS') dialog popped.

**Evidence:**




# 3 Lab 3 - Reflected XSS in JavaScript Context

**Vulnerable file:** lab3.php

**Vulnerable code:**

```
$msg = $_GET['msg'] ?? '';
...
<script>
  let msg = "$msg";
  console.log("Message: " + msg);
</script>
```

**Root cause:** User input ($_GET['msg']) is inserted directly into a JavaScript string literal with no JS-string escaping (no backslash-escaping of quotes, no JSON-encoding). This allows breaking out of the string context into live script execution.

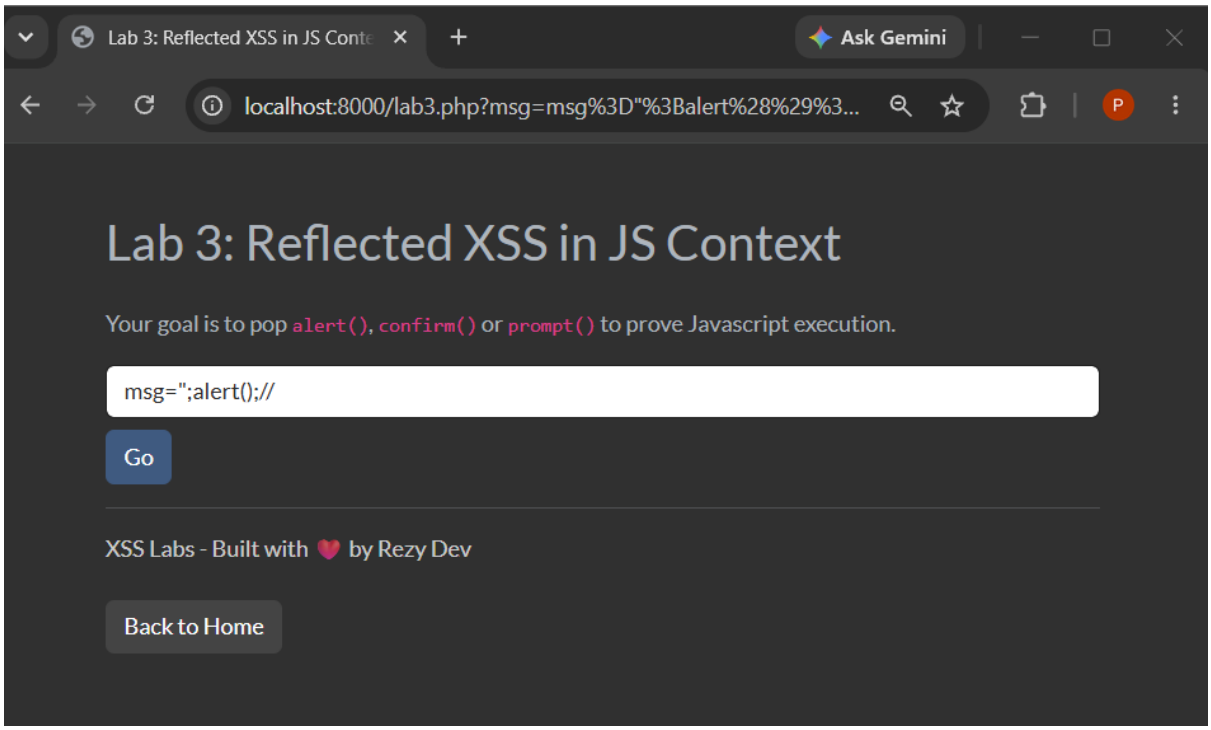
**Payload used:** msg=";alert();let x="

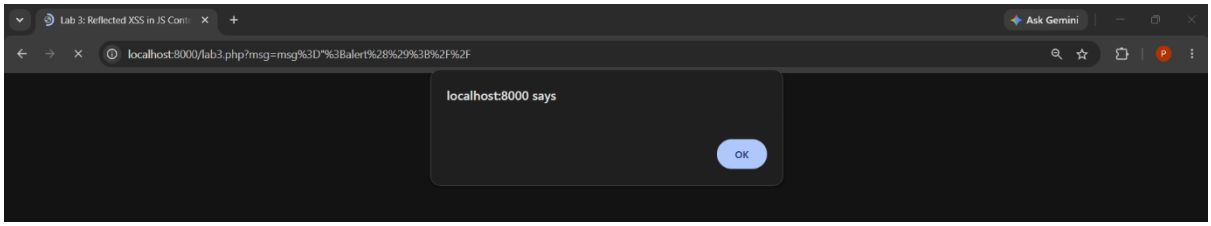
**Breakout mechanics:**

- " closes the string literal started by let msg = "
- ; terminates the statement
- alert(); is the injected JavaScript, executed immediately
- // comments out the remainder of the original line, preventing a syntax error from the trailing "; and the console.log( ..) call

**Result:** JavaScript executed successfully - alert() dialog popped on page load (no form submission needed, since it executes directly from the URL).

**Evidence:**



# 4 Lab 4 - Reflected XSS in Input Value

**Vulnerable file:** lab4.php

**Vulnerable code:**

```
$email = $_GET['email'] ?? '';
...
<input class="form-control mb-2" name="email" value="$email" placeholder="Your email">
```

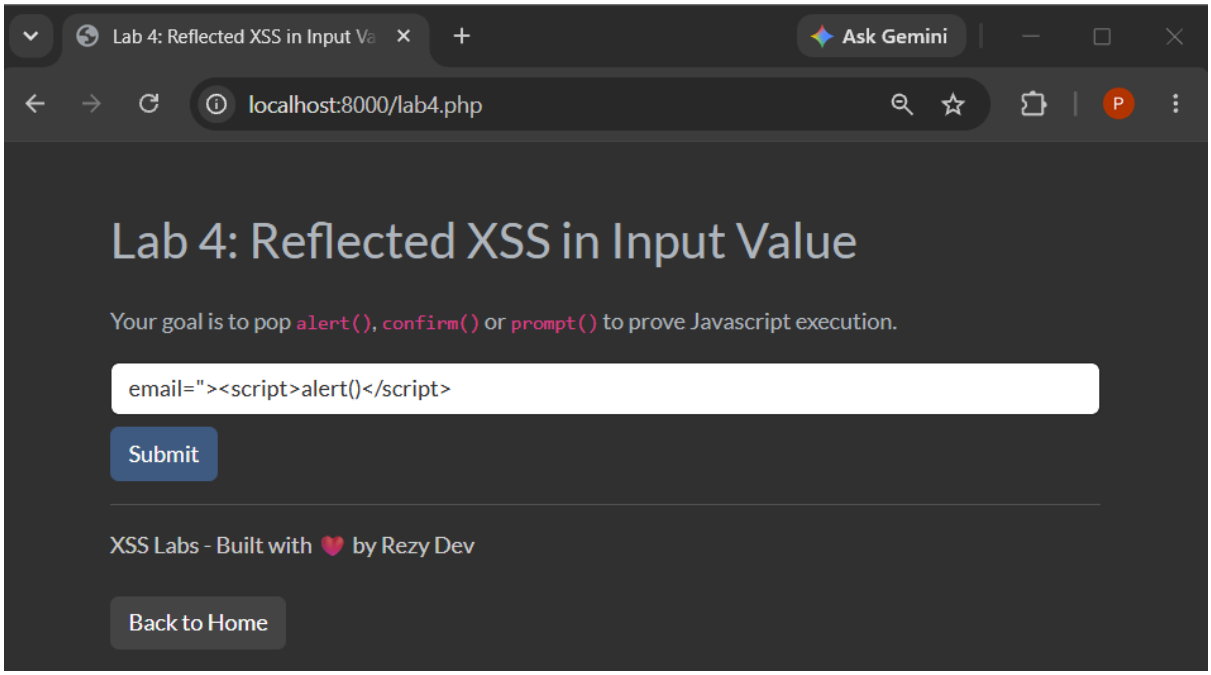
**Root cause:** User input ($_GET['email']) is reflected unencoded inside the double-quoted value attribute of an <input> tag. No htmlspecialchars() applied, allowing the attribute and tag to be broken out of.

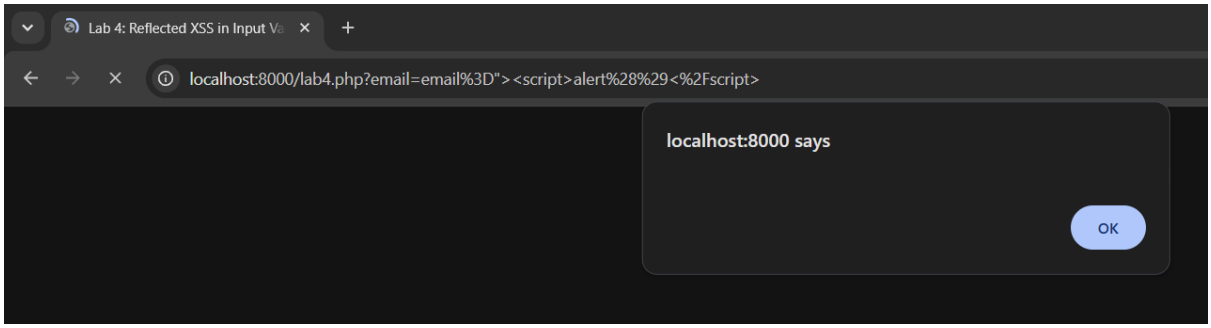
**Payload used:** "><script>alert()</script>

**Request:** GET /lab4.php?email="><script>alert()</script>

**Breakout mechanics:** " closes the value attribute → > closes the <input> tag → injected <script> element executes as part of the HTML body.

**Evidence:**






# 5 Lab 5 - Reflected XSS in URL Context

**Vulnerable file:** lab5.php

**Vulnerable code:**

```
$url = $_GET['url'] ?? '';
...
<a class="btn btn-success mt-2" href="$url">Visit site</a>
```

**Root cause:** User input ($_GET['url']) is reflected unencoded inside the double-quoted href attribute of an <a> tag. No htmlspecialchars() applied.

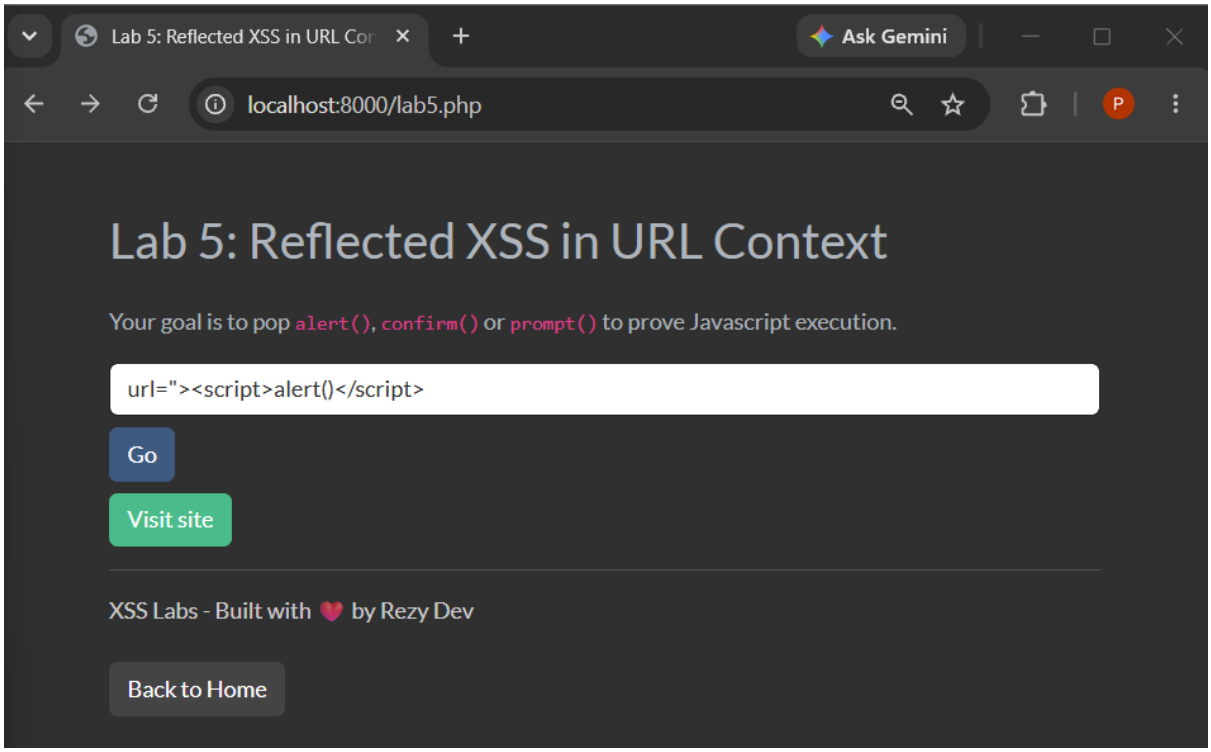
**Payload used:** "><script>alert()</script>

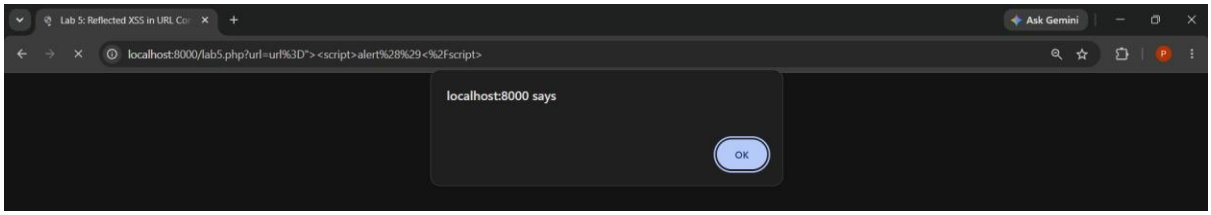
**Request:** GET /lab5.php?url="><script>alert()</script>

**Breakout mechanics:** " closes href="..." → > closes the <a> tag → injected <script> executes as part of the HTML body.

**Result:** JavaScript executed successfully — alert() dialog popped on page load.

**Evidence:**




# 6 Lab 6 - Reflected XSS with Basic Filtering (Filter Bypass)

**Vulnerable file:** lab6.php

**Vulnerable code:**

```
$comment = $_GET['comment'] ?? '';
$safe = str_replace("<script>", "", $comment);
...
<p>You said: $safe</p>
```

**Root cause:** The application attempts to sanitize input via str_replace("<script>", "", $comment), but this is a naive, case-sensitive, single-pass blacklist. It only strips the exact

lowercase string <script> and does not account for case variation, alternate tags, or event-handler-based vectors. So, the Output is otherwise unencoded.

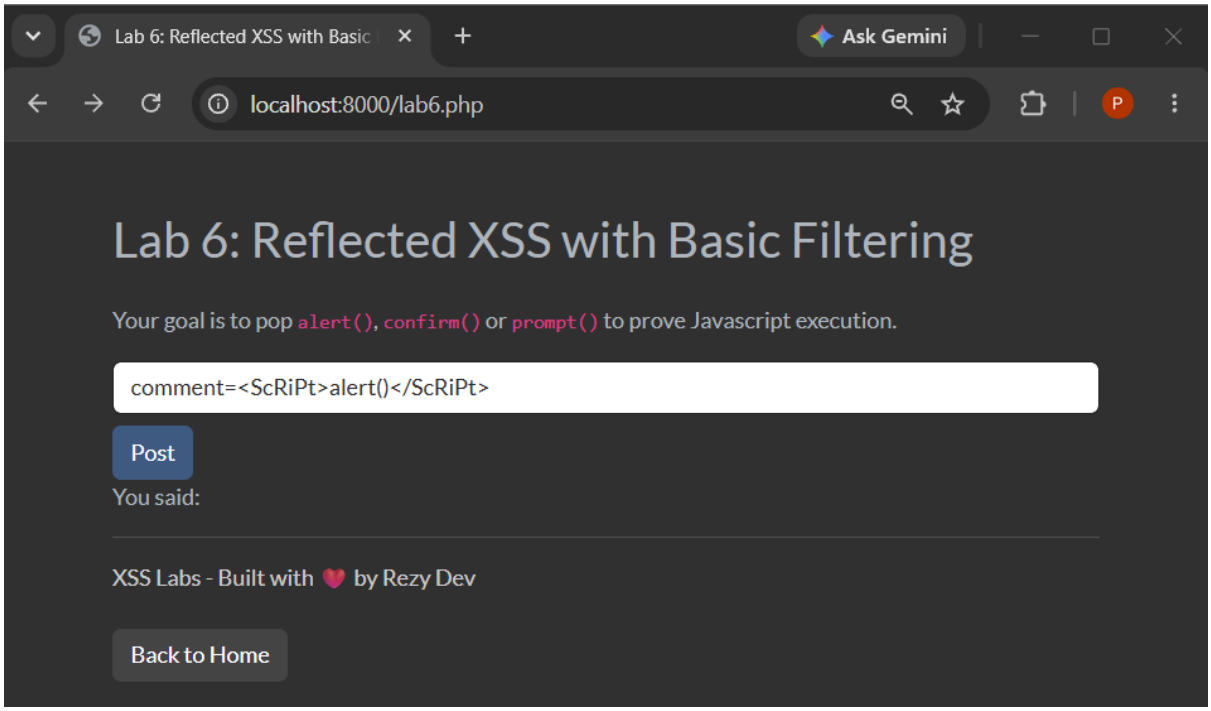
**Payload used:** <ScRiPt>alert()</ScRiPt>

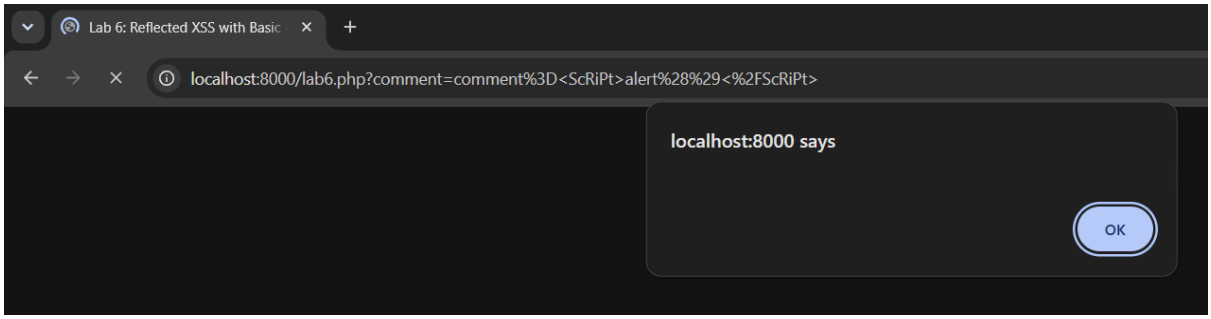
**Request**: GET /lab6.php?comment=<ScRiPt>alert()</ScRiPt>

**Bypass mechanics:** PHP's str_replace() performs an exact, case-sensitive match. Since the filter only searches for the lowercase string <script>, mixed-case variants like <ScRiPt> pass through untouched and are still interpreted by the browser as a valid <script> tag (HTML tag names are case-insensitive).

**Result:** JavaScript executed successfully — alert() dialog popped.

**Evidence:**



# 7 Lab 7 -"Impossible XSS" (Properly Mitigated)

**Vulnerable file:** lab7.php

**Code:**

```
$input = $_GET['input'] ?? '';
$safe = htmlspecialchars($input);
...
<p>Output: $safe</p>
```

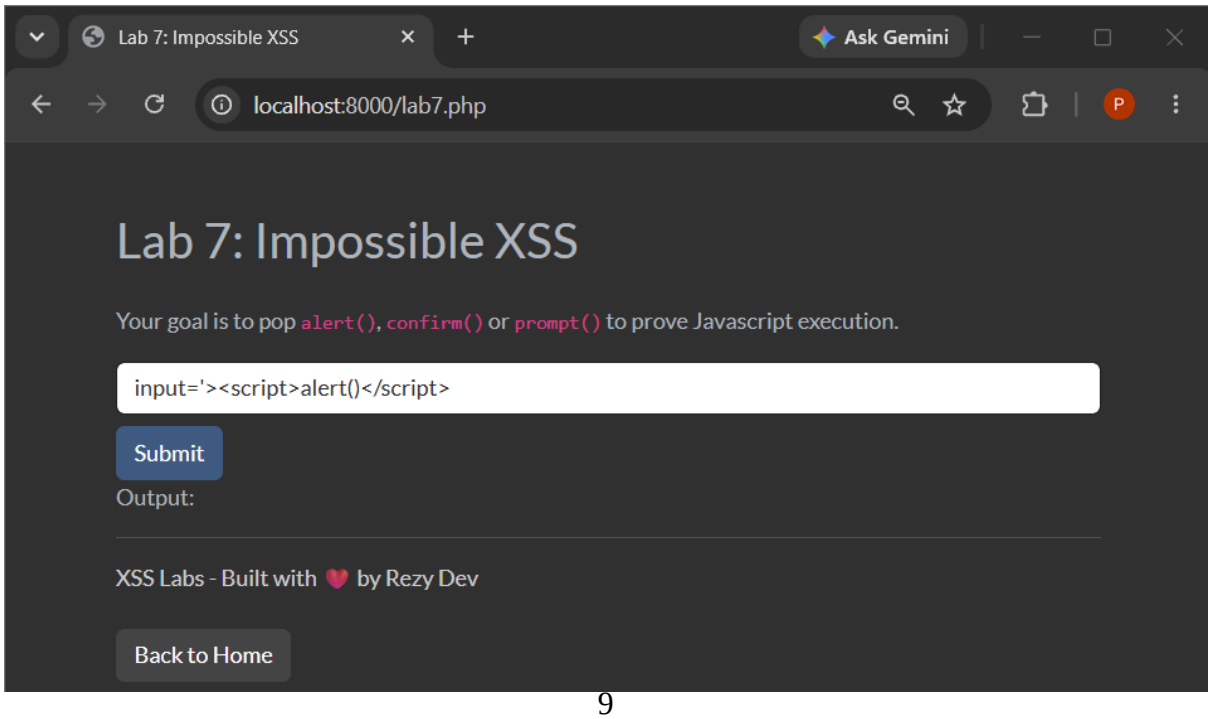
**Analysis:** Unlike previous labs, this endpoint passes user input through htmlspecialchars() before output. This function encodes HTML special characters (<, >, &, ", and — depending on PHP version/flags — ') into their HTML entity equivalents (e.g., < → <), so any injected markup is rendered as inert text rather than being parsed by the browser as HTML/JS.

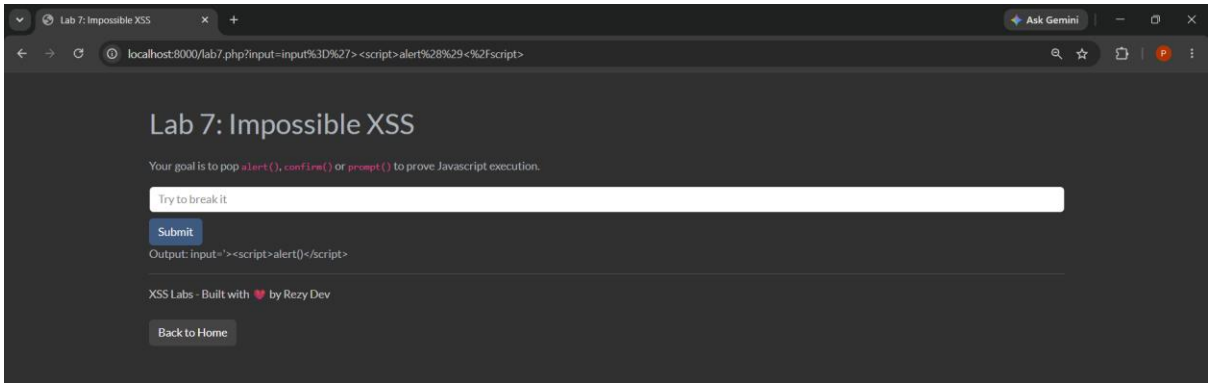
**Payload attempted:** '><script>alert()</script>

**Request:** GET /lab7.php?input='><script>alert()</script>

**Result:** No JavaScript execution. The payload was reflected back as plain, encoded text ('><script>alert()</script> visible on page but not interpreted as a tag) confirming that the output encoding neutralized the injection attempt.

**Evidence:**






# 8 Lab 8 - Reflected XSS with Weak Custom Filters (Blacklist Bypass)

**Vulnerable file:** lab8.php

**Vulnerable code:**

```
$blacklist = ['<script>', '</script>', 'onerror', 'onload', 'alert', 'svg', 'img', 'iframe', 'src='];
foreach ($blacklist as $bad) {
  $input = str_replace($bad, '', $input);
}
...
<p>Output: $input</p>
```

**Root cause:** The application relies on a hardcoded keyword blacklist to strip specific tags, event handlers, and function names. This approach is fundamentally flawed because it enumerates known-bad patterns rather than encoding output any vector not explicitly listed passes through untouched. Output is also not HTML-encoded, so any surviving markup is rendered by the browser.

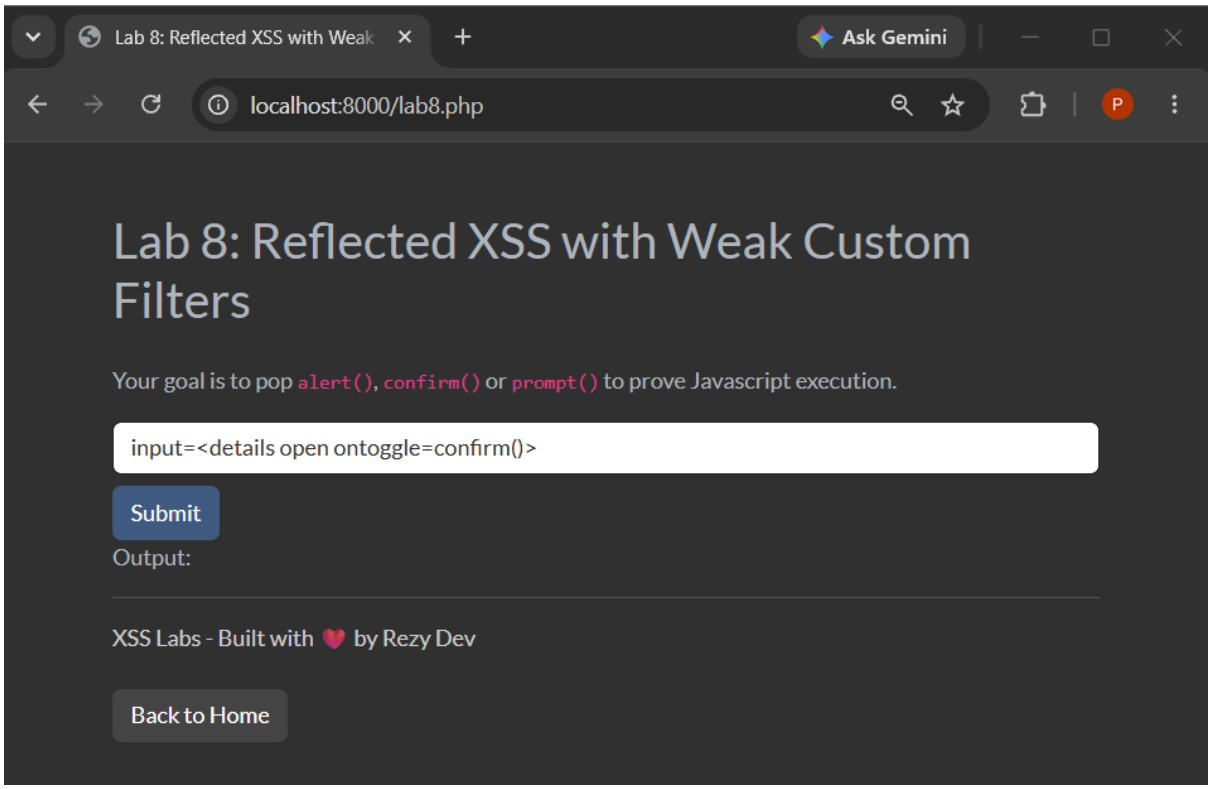
**Payload used:** <details open ontoggle=confirm()>

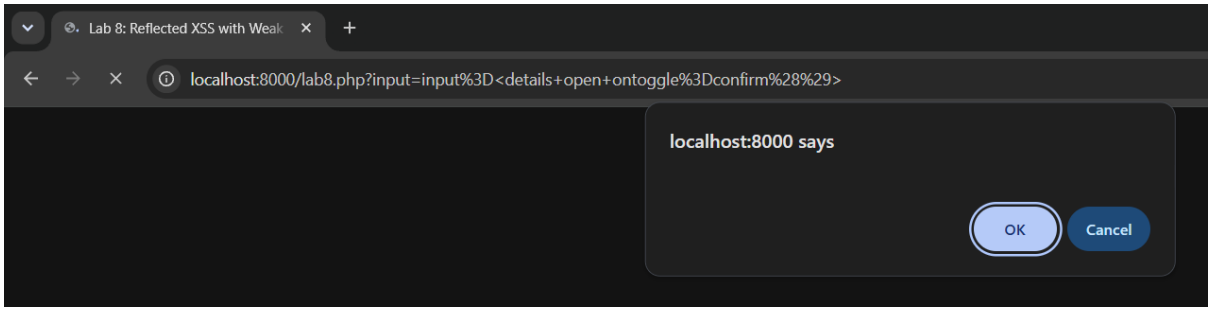
**Request:** GET /lab8.php?input=<details open ontoggle=confirm()>

**Bypass mechanics:** The blacklist only covers <script>, onerror, onload, alert, svg, img, iframe, and src=. It does not cover the <details> element or its ontoggle event, nor the confirm() function. The open attribute causes the <details> element to render expanded immediately on page load, which fires ontoggle without any user interaction, executing confirm().

**Result:** JavaScript executed successfully confirm() dialog popped (visible via the OK/Cancel button pair, distinguishing it from alert()).

**Evidence:**




# 9 Lab 9 - Reflected XSS with Moderate Custom Filters (Blacklist Bypass)

**Vulnerable file:** lab9.php

**Vulnerable code:**

```
$blacklist = ['<script>', '</script>', 'onerror', 'onload', 'alert', 'svg', 'img', 'iframe', 'src='];
foreach ($blacklist as $bad) {
  $input = str_ireplace($bad, '', $input);
}
...
<p>Output: $input</p>
```

**Root cause:** This lab upgrades Lab 8's filter from str_replace to str_ireplace, making the blacklist case-insensitive. This closes the case-variation bypass (e.g., <ScRiPt>), but the underlying design flaw remains: the filter is still a fixed keyword blacklist, not output encoding. Any tag, event handler, or JS function not explicitly listed is unaffected regardless of case. Output remains unencoded.

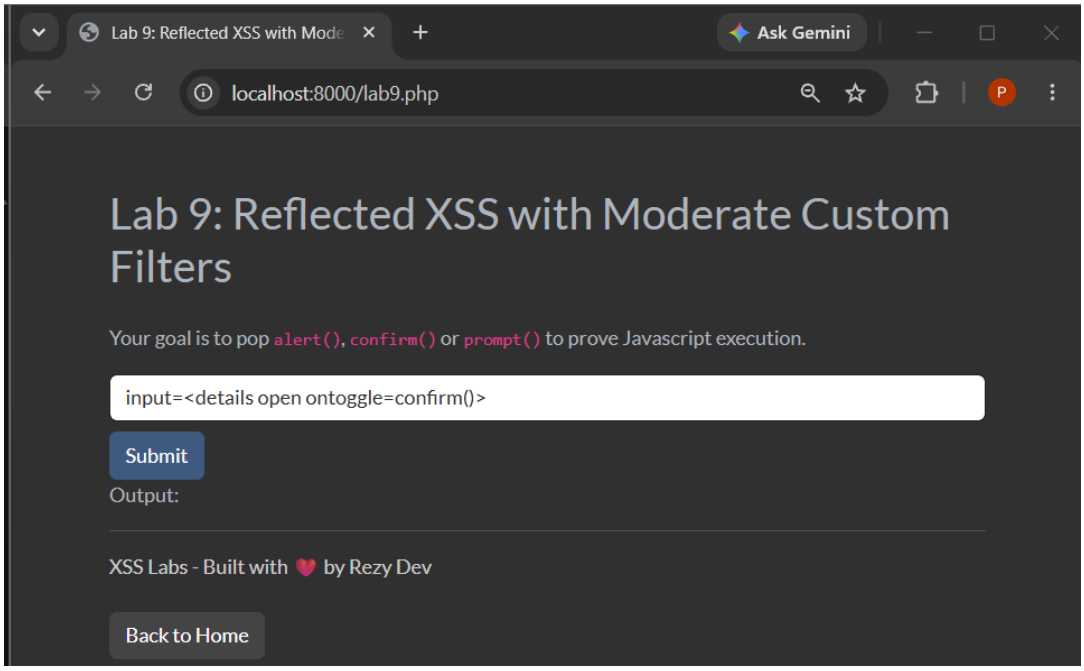
**Payload used:** <details open ontoggle=confirm()>

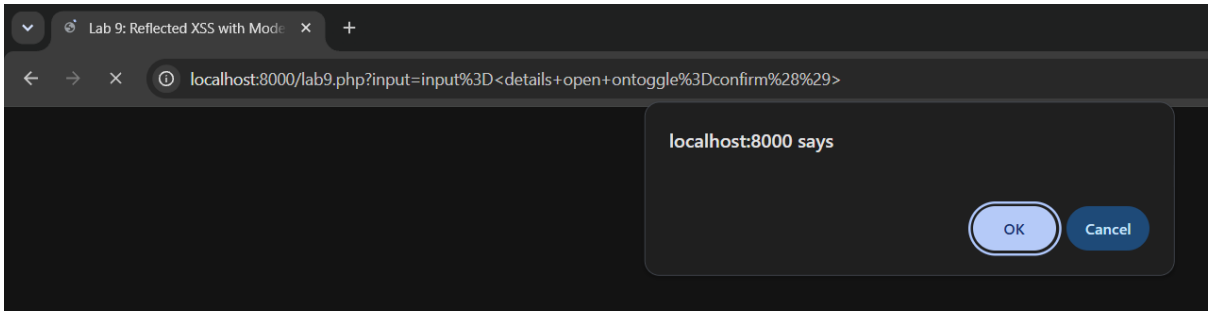
**Request:** GET /lab9.php?input=<details open ontoggle=confirm()>

**Bypass mechanics:** None of details, open, ontoggle, or confirm appear in the blacklist in any case variant, so str_ireplace has nothing to match and strip. The open attribute auto-expands the <details> element on render, firing ontoggle without user interaction.

**Result:** JavaScript executed successfully - confirm() dialog popped.

**Evidence:**






# 10 Lab 10 - Reflected XSS in JS Context with Basic Filtering

**Vulnerable file:** lab10.php

**Vulnerable code:**

```
$input = str_replace(["<", ">", "'", '"'], "", $input);
...
<script>
  let data = `$input`;
  console.log("User input:", data);
</script>
```

**Root cause:** The filter strips <, >, ", and ', which would prevent HTML tag injection or standard quoted-string breakout. However, the output sink is a JavaScript **template literal** (backtick string), and the filter does not strip backticks, $, {, or }. Template literals evaluate any ${...} expression as live JavaScript during string construction, providing a code-execution primitive that doesn't require any of the filtered characters.

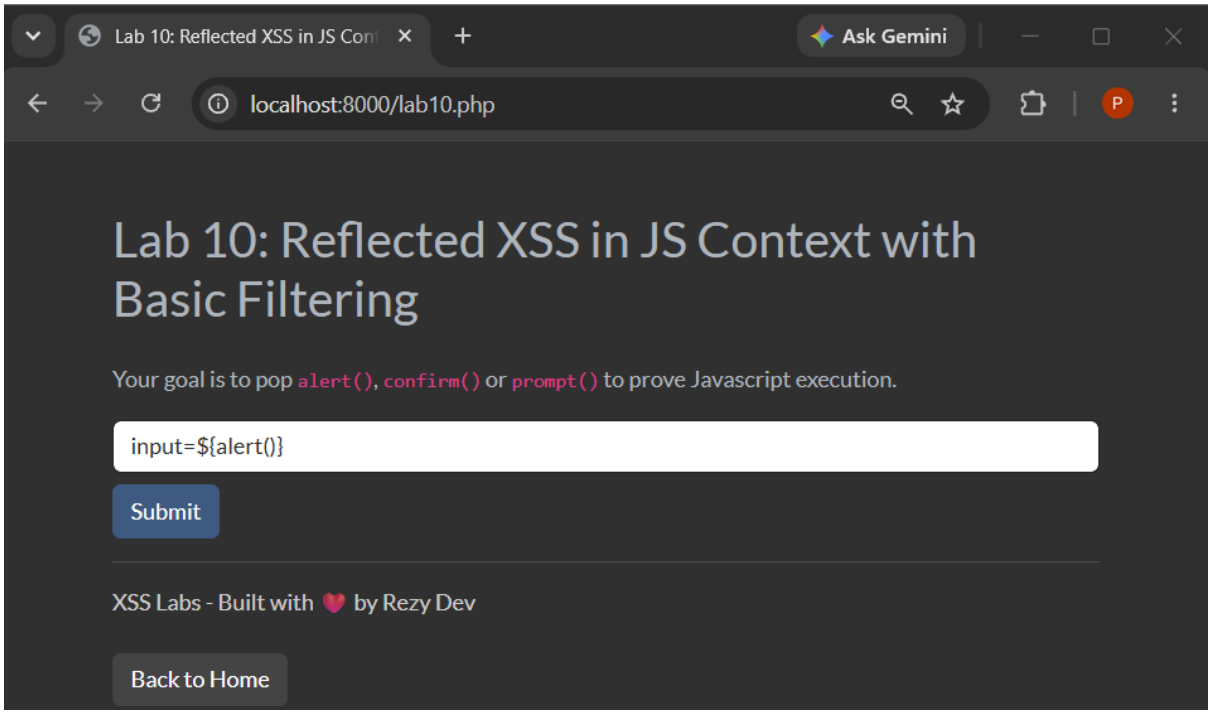
**Payload used:** ${alert()}

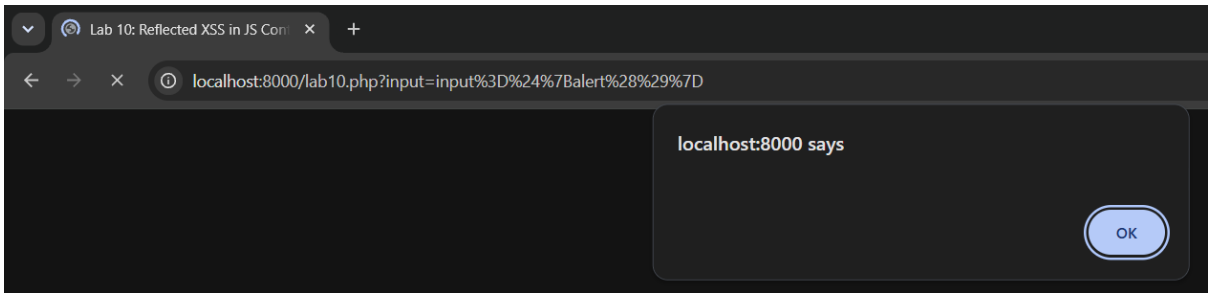
**Request:** GET /lab10.php?input=${alert()}

**Bypass mechanics:** The reflected value becomes let data = `${alert()}`;. JavaScript evaluates ${alert()} as an embedded expression inside the template literal, executing alert() immediately — no backtick, angle bracket, or quote character needed, so the character-based blacklist is entirely ineffective against this sink.

**Result:** JavaScript executed successfully — alert() dialog popped.

**Evidence:**




# 11 Lab 11 - Stored XSS

**Vulnerable file:** lab11.php

**Vulnerable code:**

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
  $msg = $_POST['msg'] ?? '';
  $_SESSION['comments_lab11'][] = $msg;
}

$comments = '';
foreach ($_SESSION['comments_lab11'] as $comment) {
  $comments .= "<li>$comment</li>";
}
...
<ul>$comments</ul>
```

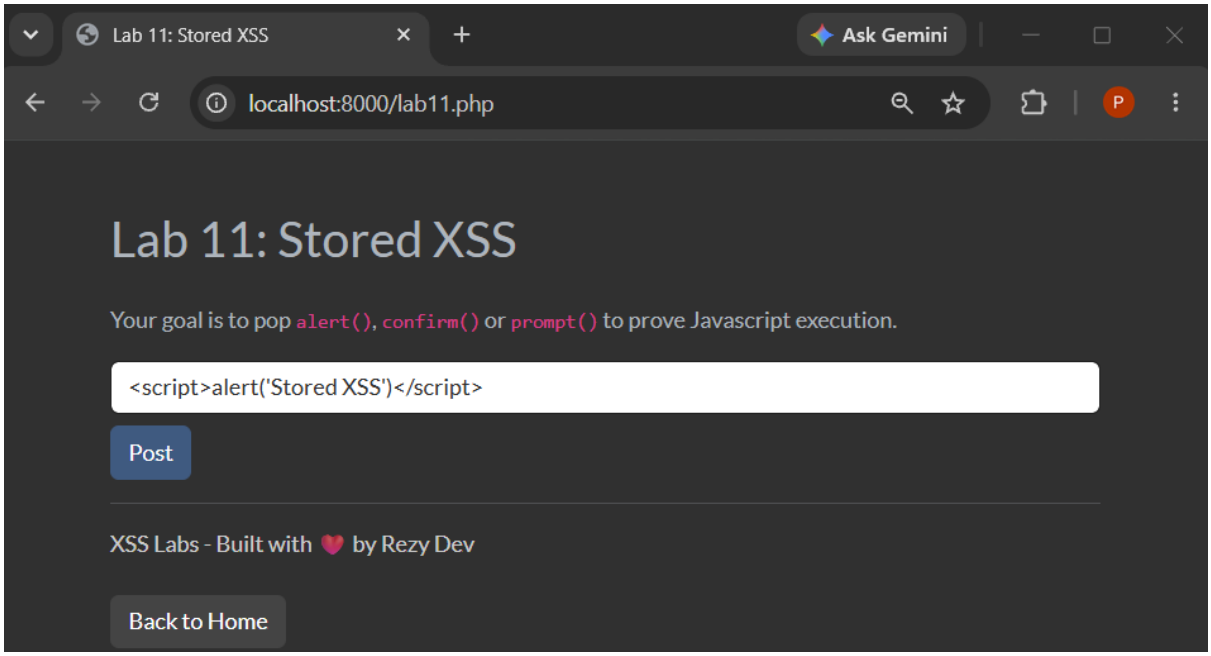
**Root cause:** User-submitted comments are stored verbatim (no sanitization, no encoding) in server-side session storage ($_SESSION['comments_lab11']), then re-rendered unencoded into <li> elements on every page load. Because the payload is persisted rather than only reflected in the immediate response, it executes for every subsequent visitor/session load not just the original request.

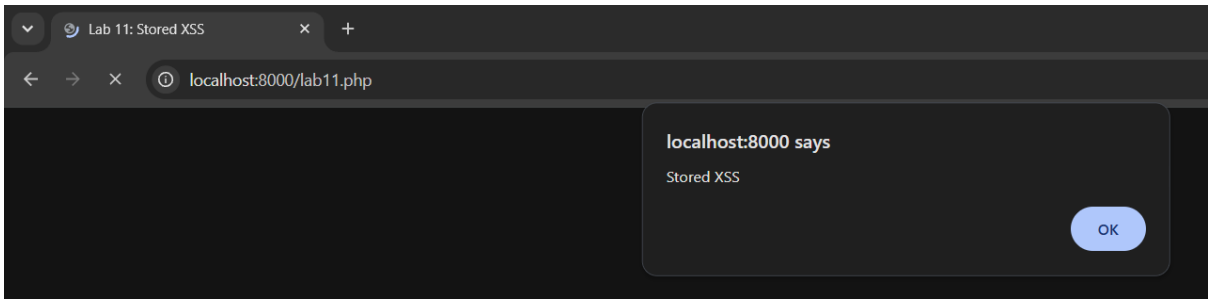
**Payload used:** <script>alert('Stored XSS')</script>

**Request:** POST /lab11.php Body: msg=<script>alert('Stored XSS')</script>

**Result:** JavaScript executed immediately after submission, and again on simple page reload (F5) with no payload present in the URL or form — confirming the payload is persisted server-side in session state rather than merely reflected in a single response.

**Evidence:**




# 12 Lab 12 - Stored XSS with Basic Filtering

**Vulnerable file:** lab12.php

**Root cause:** Same as Lab 11 (user comments stored verbatim in $_SESSION['comments_lab12'] and re-rendered unencoded into the DOM), but this time a blacklist filters <script>, </script>, onerror, onload, alert(, iframe, img, and svg. The filter is a denylist rather than proper output encoding, so it only blocks known-bad tags/attributes/function calls — it doesn't account for the many other HTML elements and event handlers that can execute JavaScript.

**Payload used:** <details open ontoggle=confirm()>

**Request:**

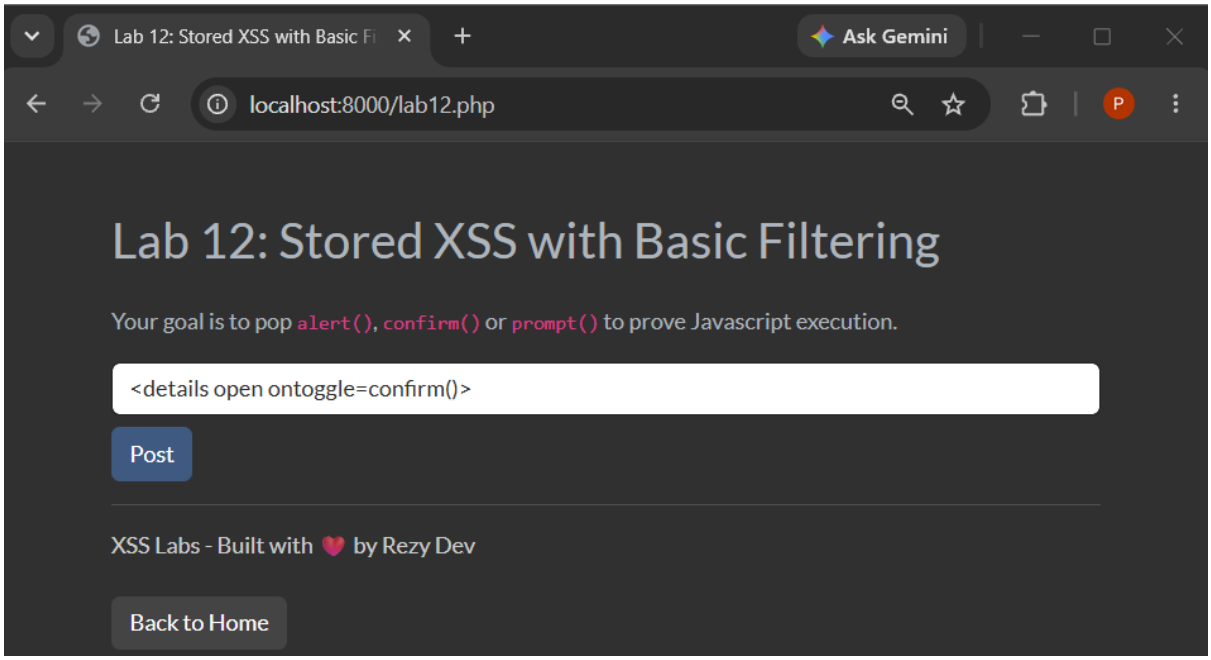
POST /lab12.php

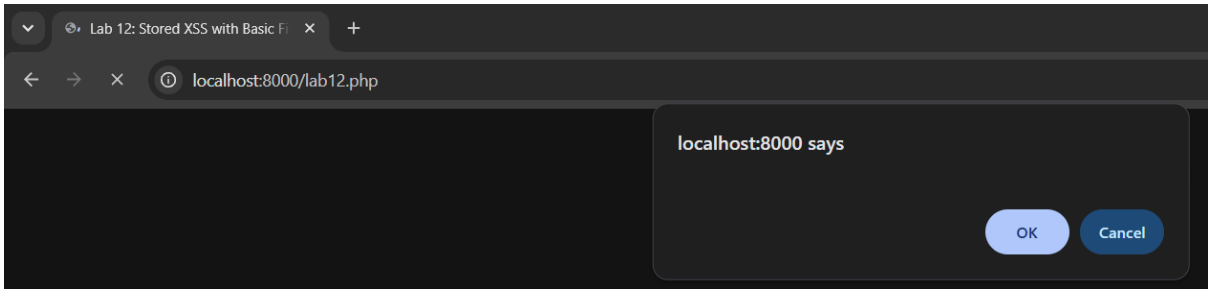
Body: msg=<details open ontoggle=confirm()>

**Why it bypasses the filter:** details, open, ontoggle, and confirm are all absent from the blacklist. The <details open> element fires its ontoggle handler automatically on render (since it's already open), so no user interaction is needed to trigger it — same reliability as the blocked onload/onerror vectors.

**Result:** confirm() dialog fired immediately after Post, and fired again on a plain page reload (F5) with no payload present in the request  confirming server-side persistence, same as Lab 11.

**Evidence:**



# 13 Lab 13 - DOM-Based XSS

**Vulnerable File:** lab13.php

**Vulnerable code:**

```
<script>
  function updateOutput() {
    let input = window.location.hash.slice(1);
    input = decodeURIComponent(input);
    document.getElementById('output').innerHTML = input;
  }

  updateOutput();

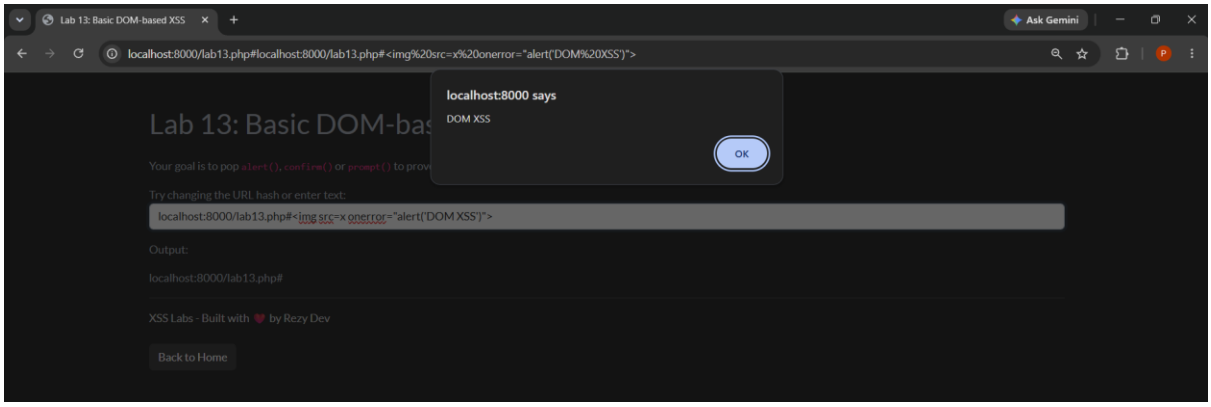
  document.getElementById('dummy').onsubmit = function(e) {
    e.preventDefault();
    let val = document.getElementById('hashInput').value;
    window.location.hash = val;
    updateOutput();
  }
</script>
```

**Root cause:** The script reads directly from window.location.hash, URL-decodes it, and assigns it straight to innerHTML with no sanitization or encoding whatsoever. This is a client-side-only sink — the payload never touches the server; the browser's own JavaScript performs the unsafe DOM write.**Payload:** <img src=x onerror="alert('DOM XSS')">

**Request:** None  delivered entirely via the URL fragment (#...), which is never transmitted to the server:

**URL**: localhost:8000/lab13.php#<img src=x onerror="alert('DOM XSS')">

**Result:** Navigating to the crafted URL caused updateOutput() to run on page load, decode the hash, and inject the payload straight into the DOM firing alert('DOM XSS') immediately, with no stored state and no server round-trip involved.**Evidence:**




## 14 Lab 14 - Advanced DOM-Based XSS

**Vulnerable file:** lab14.php

**Vulnerable code:**

```
<script>
  document.getElementById('jsonForm').onsubmit = function(e) {
    e.preventDefault();
    let raw = document.getElementById('jsonInput').value;

    try {
      let obj = JSON.parse(raw);
      document.getElementById('greeting').innerHTML = 'Hello, ' + obj.name;
    } catch {
      document.getElementById('greeting').innerText = 'Invalid JSON!';
    }
  };
</script>
```

**Root cause:** User-supplied JSON is parsed client-side with JSON.parse, which validates syntax only  it does nothing to sanitize the values inside it. The name field is then concatenated

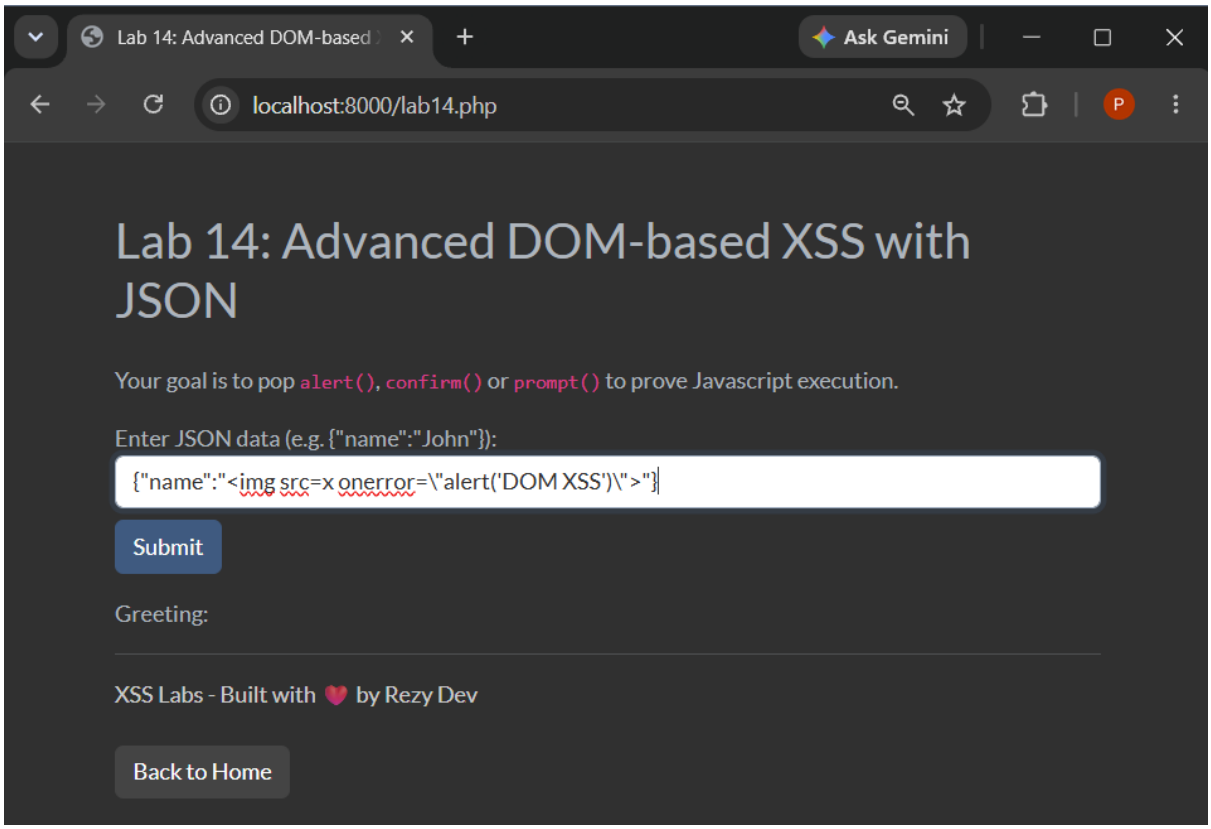
directly into innerHTML with no encoding, so any HTML/JS placed in that field is rendered live in the DOM. This is purely client-side; the payload never reaches the server.

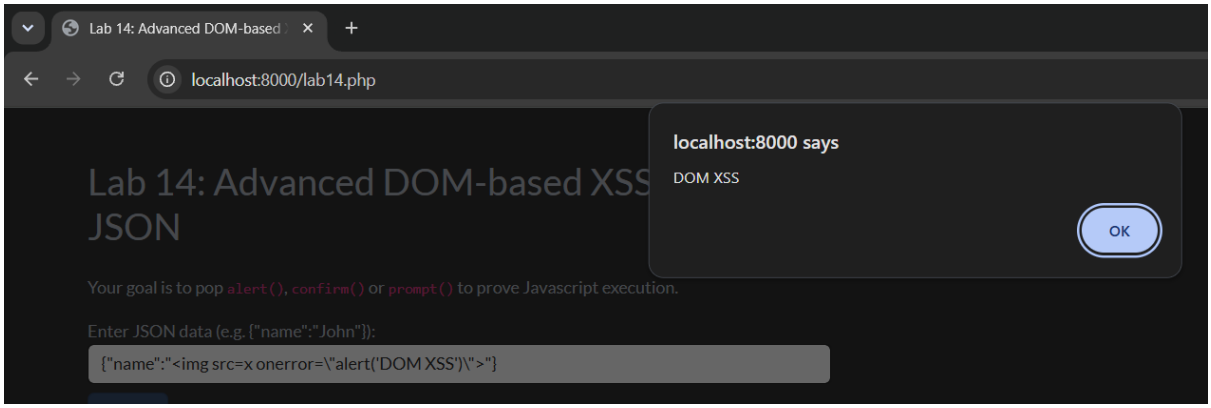
**Payload used:** {"name":"<img src=x onerror=\"alert('DOM XSS')\">"}

**Request:** None because entered into the JSON input field and submitted via the form (there is no network request; execution happens entirely client-side on submit).

**Result:** The greeting rendered "Hello," followed immediately by alert('DOM XSS') firing — confirming the unsanitized name value was injected into innerHTML and executed as HTML/JS rather than being treated as plain text.

**Evidence:**



# 15 Lab 15 - Blind XSS via Stored User Input

**Vulnerable file:** lab15.php

**Vulnerable code:**

```
if ($view === 'admin') {
    $logsHtml = '';
    foreach ($_SESSION['logs'] as $log) {
        $logsHtml .= "<li>$log</li>";
    }
    layout('Lab 15: Blind XSS - Admin Log Viewer', <<<HTML
    <h3>Admin Log Viewer</h3>
    <ul>$logsHtml</ul>
HTML);
}
```

**Root cause:** User-submitted payload values are stored verbatim no sanitization, no output encoding in $_SESSION['logs'] at submission time, then re-rendered unencoded into <li> elements only when the log is viewed from a separate, privileged context (?view=admin). This is a blind XSS: the attacker (a regular user) never sees the payload execute themselves; it only fires later, for whoever e.g: an admin views the logs. This makes it higher impact than reflected XSS in practice, since it can target privileged users with no direct interaction between attacker and victim beyond the initial submission.

**Payload used:** <script>alert('Blind XSS')</script>

**Request:**

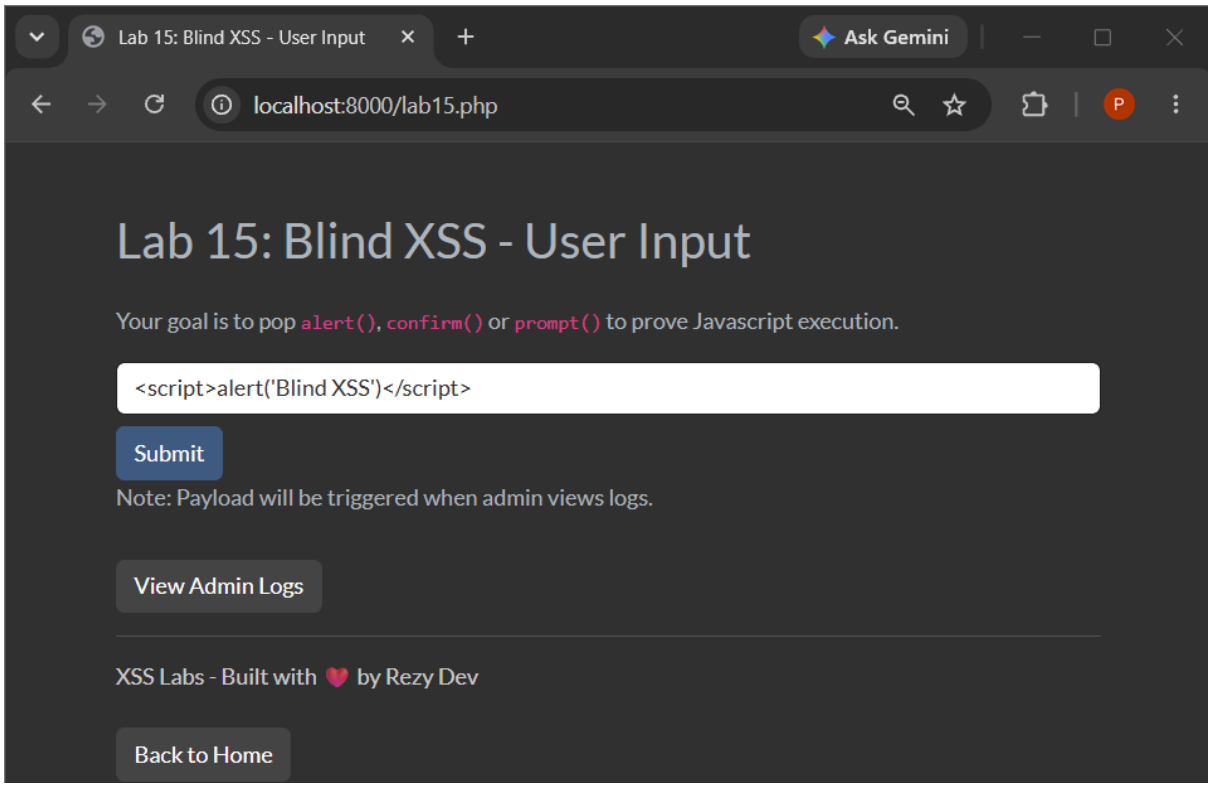
POST /lab15.php

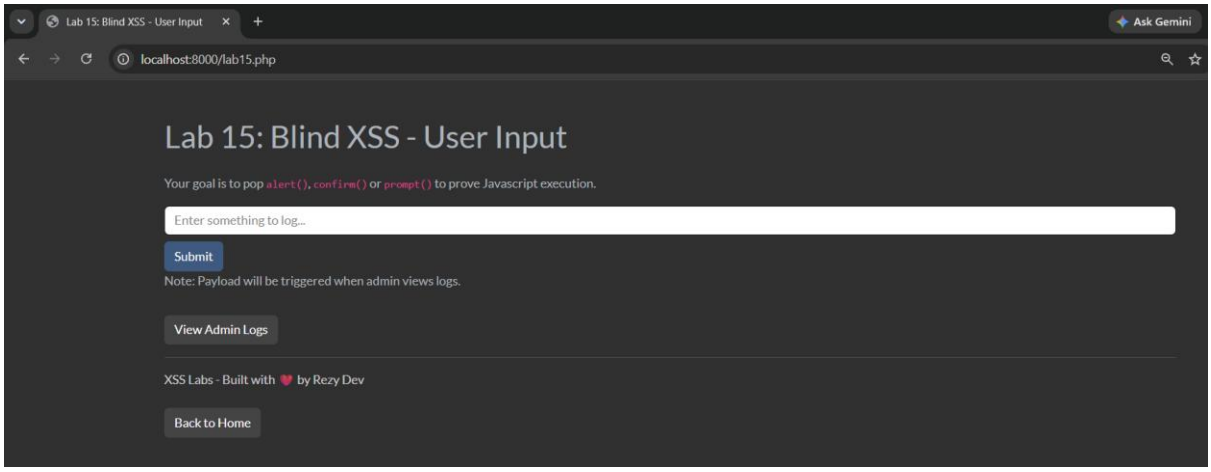
Body: payload=<script>alert('Blind XSS')</script>

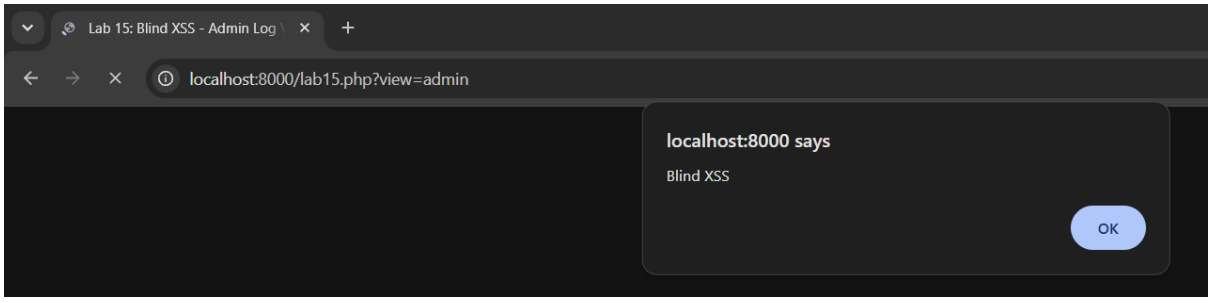
**Result:** Submitting the payload as a regular user produced no visible effect on the submission page the form simply cleared, confirming the payload doesn't execute in the attacker's own context. Navigating to /lab15.php?view=admin (simulating the admin log viewer) triggered

alert('Blind XSS') immediately, confirming the payload lay dormant in session storage until rendered in the separate admin-only view.

**Evidence:**



# 16 Lab 16 - Stored Blind XSS via User-Agent Header

**Vulnerable file:** lab16.php

**Vulnerable code:**

```
$userAgent = $_SERVER['HTTP_USER_AGENT'] ?? 'unknown';
$_SESSION['orders'][] = [
  'product' => $product,
  'userAgent' => $userAgent,
];
...
foreach ($_SESSION['orders'] as $order) {
  $orders_html .= "<li>Product: " . htmlspecialchars($order['product']) .
                  " | User-Agent: " . $order['userAgent'] . "</li>";
}
```

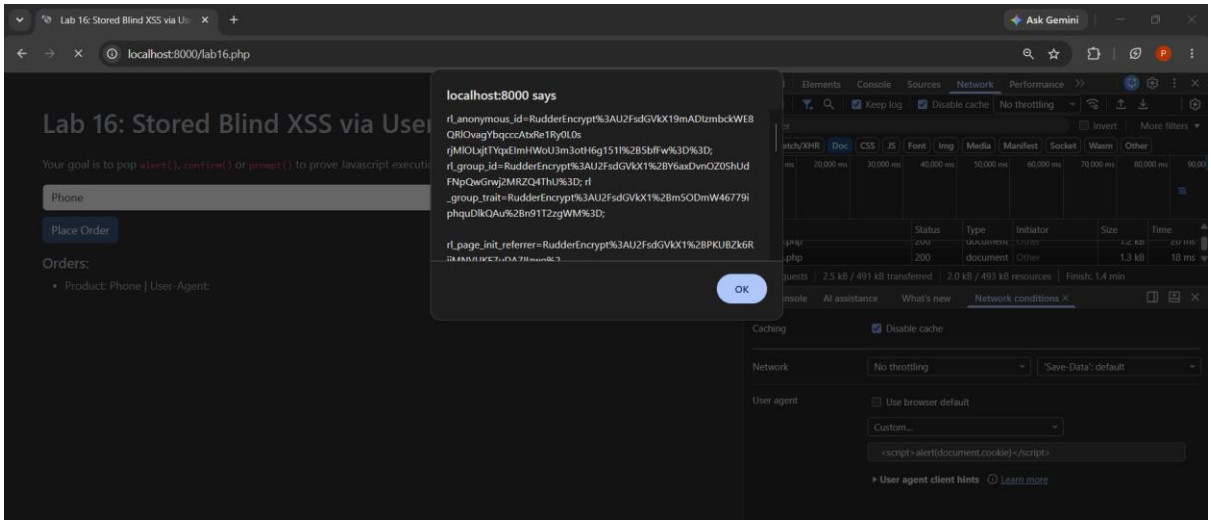
**Root cause:** The product field is properly sanitized with htmlspecialchars(), but the User-Agent HTTP request header is stored and rendered **completely unencoded**. Since the User-Agent is attacker-controlled (any HTTP client can send an arbitrary value) and not a typical form input, this is a **blind** injection vector not visible or testable through the UI, but exploitable by directly crafting the HTTP request.

**Payload used:** <script>alert(document.cookie)</script>

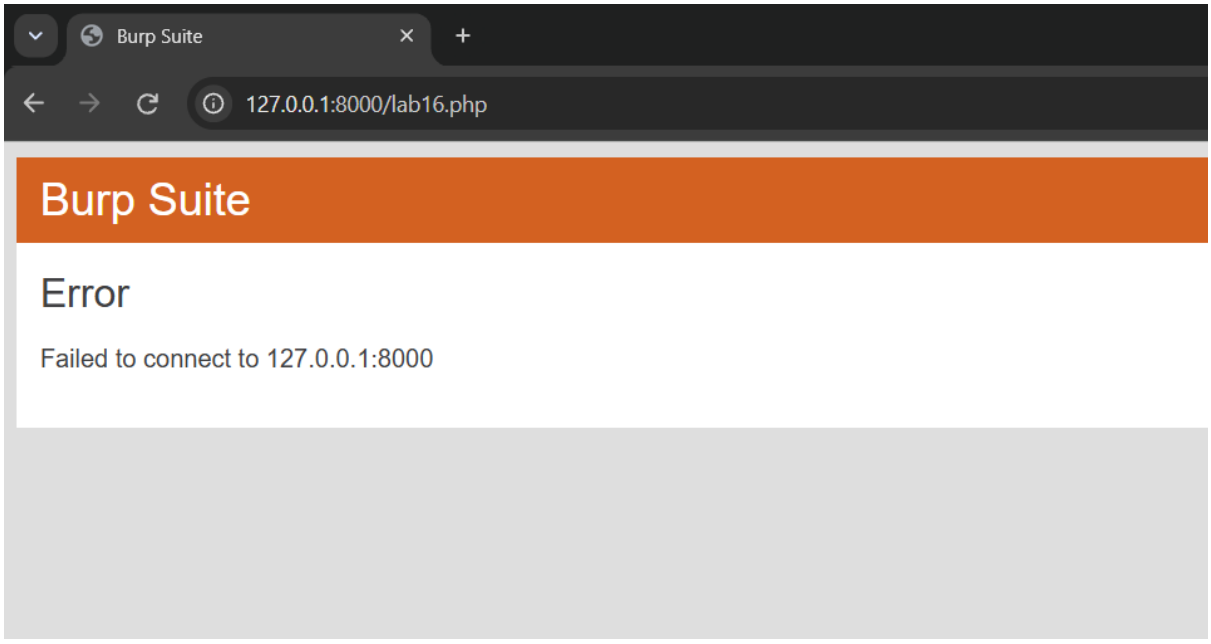
**Delivery method:** Browser DevTools -> Network conditions -> custom User-Agent override (Burp Suite Repeater was attempted first but failed due to a local connectivity/proxy issue unrelated to the application; DevTools User-Agent override was used as the practical equivalent for this environment).

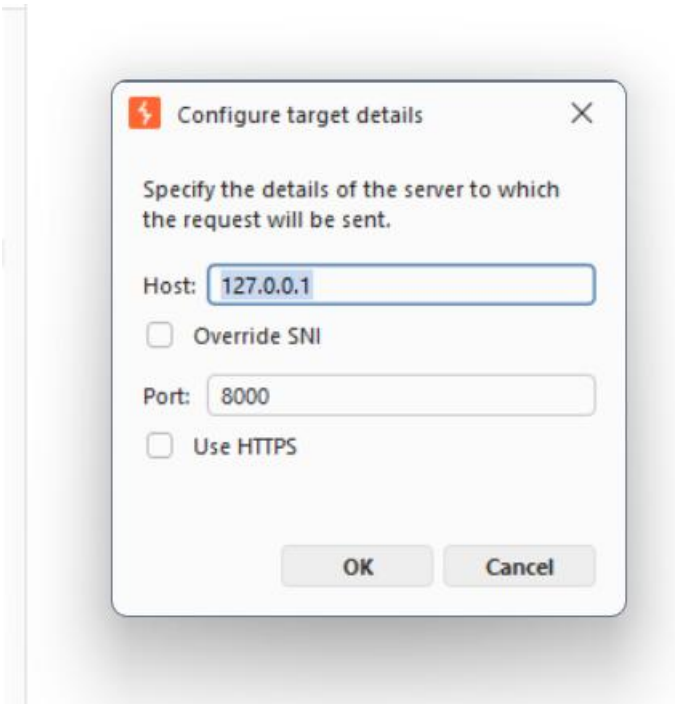
**Result:** JavaScript executed immediately on page render after order placement alert(document.cookie) fired, displaying the active session's cookies (confirming not just arbitrary JS execution, but a realistic cookie-theft impact).

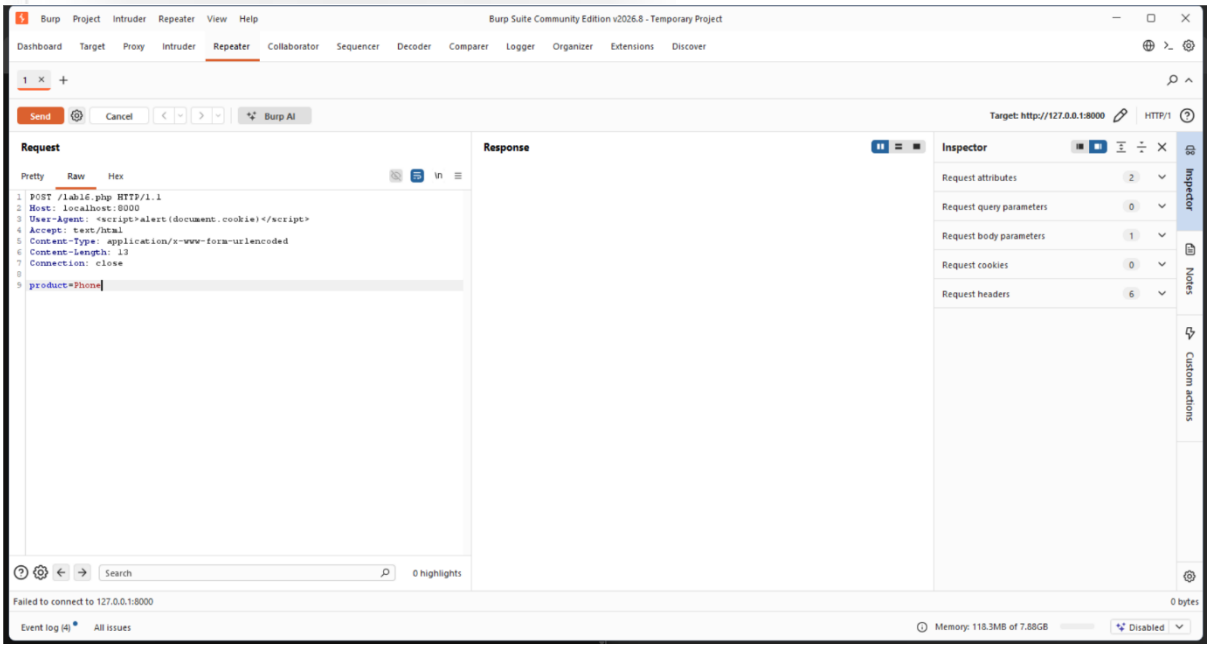
**Evidence:**

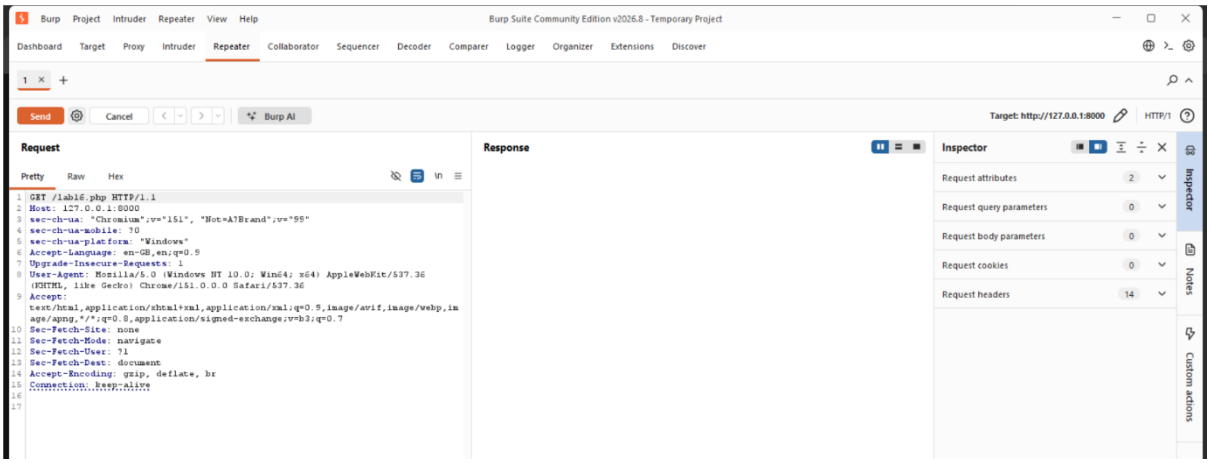



**Failed connection evidence:**




Configure target details
Specify the details of the server to which the request will be sent.
Host:
127.0.0.1
Override SNI
Port:
8000
Use HTTPS
OK
Cancel


Burp Suite Community Edition v2026.8 - Temporary Project
Request
Response
Inspector
POST /lab16.php HTTP/1.1
Host: localhost:8000
User-Agent: <script>alert(document.cookie)</script>
Accept: text/html
Content-Type: application/x-www-form-urlencoded
Content-Length: 13
Connection: close
product=Phone
Failed to connect to 127.0.0.1:8000


Burp Suite Community Edition v2026.8 - Temporary Project
Request
Response
Inspector
GET /lab16.php HTTP/1.1
Host: 127.0.0.1:8000
sec-ch-ua: "Chromium";v="151", "Not-A?Brand";v="99"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
Accept-Language: en-GB,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Accept-Encoding: gzip, deflate, br
Connection: keep-alive

## Changed to LocalHost still Failed connection

